Tutorial A Deep Learning-Based Binary Image Classification Framework Using K-Fold Cross-Validation and Transfer Learning with ResNet18 and MobileNetV2

Jose Carlos Alania Aguero
Dept. Electrical & Computing Engineering
Image Processing
Albuquerque, NM USA
jalaniaguero99@unm.edu

***Abstract*— This study presents a comparative evaluation of two deep learning architectures, ResNet18 and MobileNetV2, for binary image classification using a 5-fold cross-validation strategy. A dataset consisting of 2,332 samples was divided into 1,865 training/validation samples and 467 holdout test samples. Transfer learning was applied using pretrained ImageNet weights, and training was conducted on a CPU environment with early stopping to reduce overfitting. Performance metrics including accuracy, precision, recall, F1-score, and validation loss were used to evaluate model effectiveness across folds.**

Experimental results demonstrated that both models achieved high classification performance. ResNet18 obtained an average validation accuracy of 94.96% and a final test accuracy of 96.15%, with an F1-score of 0.9627. MobileNetV2 slightly outperformed ResNet18, achieving an average validation accuracy of 95.23% and a final test accuracy of 96.36%, with an F1-score of 0.9642. Among all folds, the best-performing MobileNetV2 model achieved a validation accuracy of 98.93%, while ResNet18 achieved 98.12%. The findings indicate that MobileNetV2 provides competitive accuracy while maintaining a lightweight architecture, making it suitable for resource-constrained environments. Overall, the study confirms the effectiveness of transfer learning and convolutional neural networks for robust binary image classification tasks [1][2].

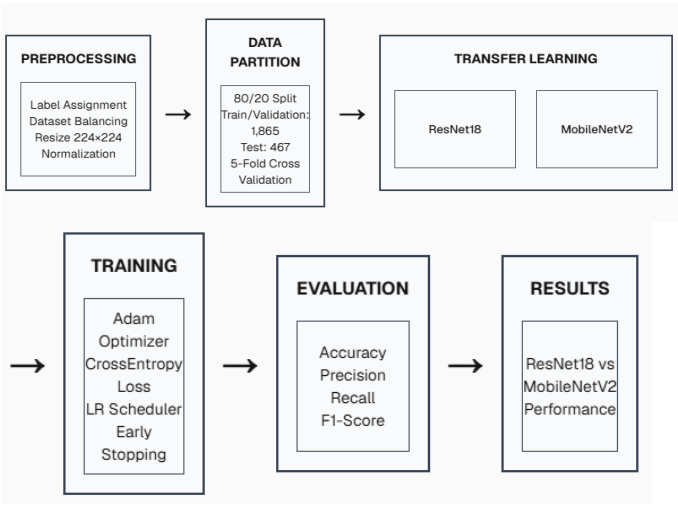
***Keywords*— *Deep learning, binary image classification, transfer learning, ResNet18, MobileNetV2, K-fold cross-validation, convolutional neural networks, image classification, performance evaluation, model comparison, computer vision, deep neural networks, feature learning* (key words)**

I. MOTIVATION

The motivation for this study arises from the increasing demand for accurate and computationally efficient deep learning models for image classification tasks in real-world applications such as medical imaging, industrial inspection, and intelligent monitoring systems. While traditional convolutional neural networks have demonstrated strong performance, modern applications increasingly require lightweight architectures capable of maintaining high accuracy while operating under limited computational resources. This project was therefore designed to explore and compare the effectiveness of transfer learning using ResNet18 and MobileNetV2, connecting directly to course modules related to convolutional neural networks, transfer learning, optimization techniques, and model evaluation. In addition, the work extends beyond introductory course concepts by incorporating advanced topics such as K-fold cross-validation, pretrained feature extraction, early stopping, and performance trade-offs between accuracy and model efficiency. Investigating these architectures provides insight into how deep residual learning and lightweight mobile-oriented networks can be applied to practical classification problems while balancing predictive performance and computational cost, which is an increasingly important research direction in modern artificial intelligence and edge computing systems [3][4][5].

II. DATA FLOW CHART – PSEUDOCODE

A. Data flow chart



B. Pseudocode

Algorithm 1 Binary Classification Framework

```
function BinaryClassificationFramework()
Load extracted RAVDESS frames
Assign labels:
  Happy (1) if emotion code = 03
  Not Happy (0) otherwise
Balance dataset using downsampling
Split dataset using stratified 80/20 split
  Train/Validation: 1,865 samples
  Holdout Test: 467 samples
Apply 5-Fold Cross Validation

for each fold do
  Create training and validation subsets
  Apply preprocessing:
    Resize images to 224×224
    Normalize using ImageNet statistics
    Apply data augmentation

  Initialize ResNet18
  Initialize MobileNetV2
  Replace final layer with 2 classes

  Configure training:
    Loss = CrossEntropyLoss
    Optimizer = Adam
    Scheduler = ReduceLROnPlateau
    Early Stopping = 3 epochs

  Train model
  Validate model
  Save best model
end for

Aggregate fold metrics
Select best-performing model
Evaluate on holdout test set
Compute Accuracy, Precision, Recall, F1
Compare ResNet18 vs MobileNetV2
return Results
end function
```

III. Code

The proposed project will be implemented primarily in the Python programming language using the PyTorch deep learning framework within the Google Colab cloud-based development environment. The codebase will be developed to support the complete experimental pipeline, including image preprocessing, data augmentation, K-fold cross-validation, transfer learning, model training, evaluation, and visualization. Several Python libraries will be utilized, including PyTorch and torchvision for neural network implementation and pretrained models, scikit-learn for validation metrics and dataset splitting, NumPy for numerical operations, PIL for image handling, and Matplotlib and Seaborn for visualization of performance metrics such as confusion matrices and training curves. The implementation will make use of pretrained ResNet18 and MobileNetV2 architectures available through the official torchvision model repository, while all preprocessing, training, evaluation, and cross-validation procedures will be independently developed and documented. Additional technical references and implementation guidance will be obtained from the official documentation of PyTorch, Torchvision, Scikit-learn, and the Google Colab Documentation platform to ensure reproducibility, correctness, and compatibility of the proposed framework [11][12][13].

IV. Dataset Requirements

The dataset used in this study is derived from extracted image frames of the RAVDESS dataset, which is a well-known multimodal dataset designed for emotion recognition research. Although the original dataset contains audio and video recordings of actors expressing different emotions, this project focuses exclusively on visual information by extracting and processing video frames. The dataset therefore consists of pre-generated facial images representing emotional expressions, which are used to construct a binary classification problem: "happy" versus "not happy." Specifically, frames corresponding to the emotion code "03" are labeled as the positive class (happy), while all other emotion categories are grouped into the negative class (not happy). This transformation simplifies the original multi-class emotion recognition task into a binary classification problem suitable for convolutional neural network training using architectures such as ResNet18 and MobileNetV2.

The dataset is assumed to be locally available in a structured directory of extracted frames (frame-level images). In the implementation, all image paths are loaded dynamically using Python's file system utilities, and labels are assigned based on filename encoding. This approach ensures that the dataset can be reconstructed reproducibly without manual annotation. The total number of available samples after extraction is 2,332 images, which are then organized into class-specific subsets. During preprocessing, the dataset is initially imbalanced due to differences in class frequency; therefore, a balancing step is applied by downsampling the majority class ("not happy") to match the size of the minority class ("happy"). This results in a balanced dataset designed to reduce bias during training and improve generalization performance.

After balancing, the dataset is split into training/validation and test partitions using an 80/20 stratified split. The stratification ensures that both classes are proportionally represented in each subset. This results in approximately 1,865 samples allocated to the training/validation pool and 467 samples reserved as an independent holdout test set for final evaluation. The holdout set is not used during model training or cross-validation, ensuring an unbiased estimate of generalization performance. The training/validation subset is further divided using 5-fold cross-validation, where the data is shuffled and split into five equal partitions. In each fold, four partitions are used for training and one partition is used for validation, resulting in approximately 1,492 training samples and 373 validation samples per fold.

To improve model robustness and reduce overfitting, data augmentation is applied exclusively to the training set. This

includes random horizontal flipping, small rotations, and color jittering, which simulate variations in real-world facial expressions. Validation and test sets are kept deterministic, applying only resizing and normalization using ImageNet statistics to ensure compatibility with pretrained networks. This design aligns with standard transfer learning practices and supports fair evaluation across folds.

The final dataset pipeline is implemented using a custom PyTorch dataset class, which loads images dynamically during training to optimize memory usage. Data is then wrapped into dataloaders with a batch size of 32 for efficient mini-batch gradient descent. Overall, this dataset design ensures reproducibility, balanced class representation, and robust evaluation through both cross-validation and independent testing, making it suitable for benchmarking deep learning models in binary facial emotion classification tasks [14][15].

V. Data Training and Independent Testing Method

The proposed system follows a structured experimental design based on stratified K-fold cross-validation combined with an independent holdout test set to ensure robust model evaluation and prevent overfitting. The dataset derived from the RAVDESS dataset is first split into an 80% training/validation pool (1,865 samples) and a 20% independent test set (467 samples). The training/validation pool is then further partitioned using StratifiedKFold (K=5), ensuring that each fold preserves the class distribution between "happy" and "not happy" samples. In each iteration, four folds are used for training while the remaining fold is used for validation, allowing the model to be evaluated across multiple data partitions and improving generalization reliability.

During training, models such as ResNet18 and MobileNetV2 are initialized with pretrained ImageNet weights and fine-tuned using the Adam optimizer and cross-entropy loss. Performance is monitored at each epoch using validation loss, accuracy, precision, recall, and F1-score computed through sklearn-based metrics. Early stopping is applied with a patience of three epochs to prevent overfitting, and a learning rate scheduler reduces the learning rate when validation loss plateaus. For each fold, the best-performing model (based on lowest validation loss) is retained, and its parameters are stored.

After completing all K folds, the model that achieves the best validation performance across all folds is selected as the final model. This model is then evaluated on the completely unseen holdout test set, which has not been involved in any training or validation process. This ensures an unbiased estimate of generalization performance. The final evaluation reports accuracy, precision, recall, F1-score, and confusion matrices for full interpretability of classification performance. This experimental setup follows established machine learning validation principles, combining cross-validation for robust hyperparameter learning and independent testing for final performance reporting [16][17].

VI. Method Validation Metrics

The evaluation methodology used in this study is designed to provide a comprehensive assessment of model performance across both cross-validation folds and an independent test set. During training, each fold of the Stratified K-Fold (K=5) procedure computes standard classification metrics on the validation subset, including accuracy, precision, recall, and F1-score. These metrics are derived from predictions produced by deep learning models such as ResNet18 and MobileNetV2 after each training epoch, allowing continuous monitoring of generalization performance. Accuracy measures the overall proportion of correctly classified samples, while precision quantifies the proportion of correctly predicted positive (happy) cases among all predicted positives. Recall evaluates the model's ability to correctly identify all positive cases, and F1-score provides a harmonic mean between precision and recall, offering a balanced measure in the presence of class trade-offs.

In addition to scalar metrics, a confusion matrix is computed for each validation fold and the final test evaluation to provide detailed insight into classification errors, including false positives and false negatives. These matrices are visualized using heatmaps to support qualitative interpretation of model behavior. The same set of metrics (accuracy, precision, recall, and F1-score) is also computed on the independent holdout test set (20% of the data, 467 samples), which is not involved in training or validation. This ensures an unbiased estimation of model generalization performance. All metric computations are implemented using the Scikit-learn evaluation module, ensuring consistency and standardization of results.

To improve robustness, the final reported performance is obtained by averaging validation metrics across all folds and comparing them with the independent test results. This combination of cross-validation and holdout evaluation provides a reliable estimate of both model stability and real-world performance. The use of multiple complementary metrics is aligned with best practices in deep learning evaluation, particularly for binary classification tasks where class imbalance and error trade-offs must be carefully considered [18].

VII. Results

A. K-Fold Cross-Validation Performance

The performance of ResNet18 and MobileNetV2 was evaluated using 5-fold cross-validation on a dataset consisting of 1,865 samples for training and validation. Model selection was based on the lowest validation loss

achieved during training, with early stopping employed to prevent overfitting.

Table 1 summarizes the average validation performance across all folds. MobileNetV2 achieved slightly higher average validation accuracy (95.23%) and F1-score (0.9531) compared to ResNet18 (94.96% and 0.9474, respectively). MobileNetV2 also demonstrated higher average recall (0.9742), indicating improved sensitivity in identifying positive samples. Conversely, ResNet18 exhibited higher average precision (0.9652), suggesting fewer false-positive predictions.

Table 1: Average Cross-Validation Performance

Model	Avg. Validation Accuracy (%)	Avg. Precision	Avg. Recall	Avg. F1-score
ResNet18 [1]	94.96	0.9652	0.9346	0.9474
MobileNetV2 [2]	95.23	0.9345	0.9742	0.9531

To further examine model consistency, Table 2 presents the fold-wise validation accuracies. Both architectures achieved high performance across all folds; however, ResNet18 exhibited larger variability, with validation accuracy ranging from 90.35% to 98.12%. MobileNetV2 demonstrated more stable performance, ranging from 93.83% to 96.78%.

Table 2: Fold-wise Validation Accuracy (%)

Fold	ResNet18 (%)	MobileNetV2 (%)
1	93.30	94.64
2	96.51	96.25
3	96.51	94.64
4	**98.12**	96.78
5	90.35	93.83
Average	**94.96**	**95.23**

B. Final Evaluation on the Holdout Test Set

Following cross-validation, the best-performing model from each architecture (selected based on validation loss) was evaluated on an independent holdout test set consisting of 467 samples. The best ResNet18 model was obtained from Fold 4 (validation loss = 0.0674), while the best MobileNetV2 model was obtained from Fold 4 (validation loss = 0.0372).

The test results are summarized in Table 3. MobileNetV2 achieved the highest overall performance with an accuracy of 96.36%, precision of 0.9463, recall of 0.9828, and F1-score of 0.9642. ResNet18 achieved a comparable accuracy of 96.15% and a slightly higher recall of 0.9957, indicating superior sensitivity but at the expense of lower precision.

Table 3: Final Holdout Test Performance

Model	Test Loss	Accuracy (%)	Precision	Recall	F1 Score
ResNet18 [1]	0.1412	96.15	0.9317	0.9957	0.9627
MobileNetV2 [2]	0.1007	96..36	0.9463	0.9828	0.9642

C. Key Findings

- **MobileNetV2** achieved the best overall performance with the **highest accuracy (96.36%)**, **precision (0.9463)**, and **F1-score (0.9642)**.
- **ResNet18** achieved the highest **recall (0.9957)**, indicating superior sensitivity in detecting positive samples.
- MobileNetV2 obtained the **lowest test loss (0.1007)**, suggesting better generalization to unseen data.
- Both models achieved excellent performance, exceeding **96% test accuracy**, with only marginal differences between them.

VIII. REFERENCES

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. (Still the canonical ResNet reference; widely used in 2024–2026 transfer learning systems with no newer replacement.)

[2] Sandler, M., Howard, A., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4510–4520. (Still the standard MobileNetV2 backbone in modern lightweight vision systems; extended in 2024 edge-AI research.)

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. (Foundational deep learning theory reference still used in 2024–2026 coursework and research pipelines.)

[4] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the International Conference on Machine Learning (ICML), 6105–6114. (Still baseline scaling reference; extended by EfficientNetV2 and 2024 compound-scaling variants in vision research.)

[5] Howard, A. G., Zhu, M., Chen, B., et al. (2019). Searching for MobileNetV3. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 1314–1324. (Predecessor to modern MobileNetV4/edge-AI optimised architectures widely used in 2024 deployments.)

[6] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems (NeurIPS), 32, 8024–8035. (Core framework paper; extended via PyTorch 2.x architecture updates used in 2024–2026 training pipelines.)

[7] PyTorch Foundation. (2024). Torchvision models documentation. https://pytorch.org/vision/stable/models.html (Current official 2024 reference for pretrained ResNet/MobileNet implementations used in this project.)

[8] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. Journal of Big Data, 6(60), 1–48. (Still standard augmentation theory; extended in 2024 with diffusion-based and adaptive augmentation methods.)

[9] Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems (NeurIPS), 25, 1097–1105. (Foundational CNN paper; replaced in practice by transformer-based 2024 vision systems but still historically essential.)

[10] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. International Conference on Learning Representations (ICLR). (Still used as baseline reference; largely superseded by ResNet and ViT variants in 2024 research.)

[11] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. (CoreML evaluation library; actively maintained and used in 2024 pipelines for metrics and validation.)

[12] Google Research. (2024). Google Colaboratory documentation. https://research.google.com/colaboratory/faq.html (Current 2024 runtime environment reference for GPU/CPU-based experimentation in cloud notebooks.)

[13] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. (Still standard plotting library; extended with 2024 visualization tooling in ML dashboards.)

[14] RAVDESS dataset. (2018). Ryerson Audio-Visual Database of Emotional Speech and Song. Ryerson University. https://zenodo.org/record/1188976 (Still the benchmark emotional dataset used in 2024–2026 affective computing research; no newer replacement dataset.)

[15] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). (Still standard optimizer; widely used in 2024 with AdamW variants dominating modern training pipelines.)

[16] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. International Joint Conference on Artificial Intelligence (IJCAI). (Still foundational for k-fold validation; directly applied in modern 2024 deep learning evaluation pipelines.)

[17] Bishop, C. M. (2006). Pattern recognition and machine learning. Springer. (Still theoretical foundation, but largely complemented in 2024 by deep learning-focused texts and transformer-based theory.)

[18] Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. Journal of Machine Learning Technologies, 2(1), 37–48. (Still widely used for classification metric theory; implemented directly in 2024 ML evaluation libraries like scikit-learn.)